

Sharding the Optimizer State

Stop storing the same numbers on every worker · video 2:08

This lesson answers one question: why should every data-parallel worker store the same Adam state? It shouldn't. With **optimizer state sharding** (ZeRO, FSDP), each worker keeps only its slice: it updates its slice of the weights, then the workers **all-gather** the result. A real run with PyTorch's ZeroRedundancyOptimizer gives identical losses with a quarter of the Adam state per worker; sharding weights and gradients too brings a 7-billion- parameter model from 112 GB to under 2 GB per worker on 64 workers.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 5).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. Every worker, the same state	▶ 0:08-0:38	1	☐
2. Same training, a quarter of the state	▶ 0:38-1:06	2	☐
3. At scale, and the cost	▶ 1:06-1:58	3	☐

Before you start: Deep Dive episodes 19 (AdamW) and 25 (data parallelism). Code: `code/d27_sharding` (PyTorch only; starts 4 local processes; a few minutes on a CPU).

CONCEPT 1 OF 3

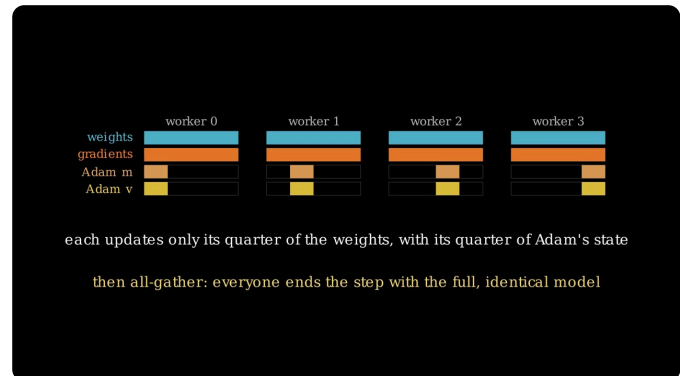
Every worker, the same state

▶ Video 0:08–0:38

Data parallelism has a wasteful habit: every worker stores the same Adam state, the same two numbers per parameter, copied on every machine. **Sharding** gives each worker only its own slice. After the gradients are averaged, each worker updates only its slice of the weights, using only its slice of Adam's state; then the workers share the updated slices with an **all-gather**, so everyone ends the step with the full, identical model.



▶ 0:20 Data parallelism copies weights, gradients and Adam's m and v on every worker.



▶ 0:37 Sharded: each worker updates its quarter with its quarter of Adam's state, then all-gather.

KEY IDEA

Each optimizer state entry only needs to live on one worker; the workers coordinate to rebuild the full weights after the update.

Check 1: Every worker, the same state

Answer all 1 questions before looking at the answers. Pass mark: all correct.

1.1 SHORT ANSWER

Why can each worker update only its own slice of the weights correctly?

.....

CONCEPT 2 OF 3

Same training, a quarter of the state

▶ Video 0:38–1:06

Four processes train the same 818,241-parameter tiny GPT for 100 steps, once with plain AdamW and once with PyTorch's `ZeroRedundancyOptimizer` wrapping AdamW. The losses are identical (largest difference 0.0; **2.4467** after 100 steps in both runs). But each worker holds **6.24 MiB** of Adam state with the plain optimizer and **1.56 MiB** sharded: a quarter, with four workers.

4 processes, 100 steps: plain AdamW vs ZeroRedundancyOptimizer

largest loss difference: 0.0
step 100: 2.4467 vs 2.4467

▶ 0:53 4 processes, 100 steps: plain AdamW and ZeroRedundancyOptimizer give identical losses (2.4467).

Adam state held by each worker (818,241 parameters)

plain 6.24 MiB
sharded 1.56 MiB
a quarter, with four workers

▶ 1:05 Adam state per worker: 6.24 MiB plain, 1.56 MiB sharded.

KEY IDEA

Sharding changes where numbers are stored, not what is computed.

Check 2: Same training, a quarter of the state

Answer all 2 questions before looking at the answers. Pass mark: all correct.

2.1 CALCULATION

With 8 workers, how much Adam state would each hold for the tiny model?

2.2 TRUE OR FALSE

“Sharding the optimizer state changes the training result.”

True ☐ False ☐

CONCEPT 3 OF 3

At scale, and the cost

▶ Video 1:06–1:58

At scale this is what makes training possible. A 7-billion-parameter model with Adam in float32 (16 bytes per parameter) on 64 workers: everything copied needs **112 GB** per worker; sharding Adam's state (8 of the 16 bytes) brings it to **56.9 GB**; sharding the weights and gradients too, **1.8 GB** (activations not counted).

Nothing is free: sharded weights must be gathered just before each layer uses them and released afterwards, so communication is traded for memory. You will meet these ideas as **ZeRO**, from Microsoft's DeepSpeed, and as **FSDP**, fully sharded data parallel, in PyTorch. In code, sharding the optimizer is one line: wrap AdamW in `ZeroRedundancyOptimizer`; the training loop doesn't change.

7 billion parameters, Adam in float32, 64 workers: memory per worker

everything copied 112.0 GB

Adam state sharded 56.9 GB

weights, gradients and state sharded 1.8 GB

ZeRO (Microsoft DeepSpeed)

FSDP: fully sharded data parallel (PyTorch)

no worker stores what another already holds

▶ 1:24 7B parameters on 64 workers: 112 GB copied, 56.9 GB with Adam state sharded, 1.8 GB with everything sharded.

▶ 1:48 ZeRO (Microsoft DeepSpeed) and FSDP (PyTorch): no worker stores what another already holds.

KEY IDEA

Shard the optimizer state, then the gradients, then the weights: each step saves memory and adds communication.

Check 3: At scale, and the cost

Answer all 3 questions before looking at the answers. Pass mark: all correct.

3.1 CALCULATION

GPT-2 small (124,439,808 parameters, 16 bytes each) fully sharded over 8 workers: how much per worker?

3.2 CALCULATION

For 7B parameters on 64 workers, show where 56.9 GB comes from.

3.3 MULTIPLE CHOICE

What does full sharding (weights too) cost?

- A. Lower accuracy
- B. More communication: weights gathered before each layer
- C. More memory
- D. Nothing

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. Every worker, the same state · rewatch ▶ 0:08-0:38

1.1 After the all-reduce, every worker has the same averaged gradients; Adam's update for each weight uses only that weight's gradient and its own m and v, so a slice can be updated independently. Adam is element-wise: no weight's update depends on another weight's state.

2. Same training, a quarter of the state · rewatch ▶ 0:38-1:06

2.1 About 0.78 MiB. $6.24 \text{ MiB} / 8$.

2.2 False. The episode's losses were identical to every printed digit.

3. At scale, and the cost · rewatch ▶ 1:06-1:58

3.1 About 249 MB. $124,439,808 \times 16 / 8 \approx 2.49 \times 10^8$ bytes.

3.2 $8 \text{ bytes} \times 7 \times 10^9$ (weights + gradients, full) = 56 GB, plus $8 \text{ bytes} \times 7 \times 10^9 / 64$ (Adam's share) $\approx 0.875 \text{ GB}$. Only m and v are divided among the workers.

3.3 B. Memory is traded for traffic.